



Gruppo SkyNet
skynetunigroup@gmail.com

Analisi dei Requisiti

Versione	<i>0.14.0</i>
Uso	<i>Esterno</i>
Destinatari	<i>Prof. Tullio Vardanega, Prof. Riccardo Cardin</i>
Redattori	<i>Sara Ristovic, Marco Barbiero, Dario Pirrone</i>
Verifica	<i>Dario Pirrone, Marco Barbiero, Samuele Bertin, Edoardo Granziero</i>
Approvazione	<i>Samuele Bertin, Dario Pirrone, Andrea Fistarol, Edoardo Granziero, Sara Ristovic</i>

Versioni del documento

Ver.	Data	Descrizione	Autore	Verificatore
0.14.0	2026/06/18	Aggiunti UC e requisiti relativi alla registrazione e login	Dario Pirrone	Sara Ristovic
0.13.0	2026/06/08	Aggiunti diagrammi UML per Agente OWASP e Agente Changelog	Dario Pirrone	Edoardo Granziero
0.12.1	2026/06/06	Modificata numerazione UC e requisiti	Dario Pirrone	Edoardo Granziero
0.12.0	2026/06/05	Modifiche varie a UC e ai requisiti a seguito di incontro esterno con VarGroup	Dario Pirrone	Edoardo Granziero
0.11.0	2026/05/30	Uniti UC 7, 7.1, 7.2; 8, 8.1, 8.2, 8.3; 9, 9.1, 9.2. ex UC 10 integrato in UC 7, 8, 9. Modifiche al tracciamento.	Andrea Fistarol	Edoardo Granziero
0.10.0	2026/05/30	Aggiunti requisiti RF.20 e RQ.4 e modificata sezione 8	Dario Pirrone	Edoardo Granziero
0.9.0	2026/05/28	Scrittura della sottosezione "Attori" (3.2)	Dario Pirrone	Edoardo Granziero
0.8.2	2026/05/26	Raffinati casi dell'agente Docs e adeguati gli UC al modello	Andrea Fistarol	Samuele Bertin
0.8.0	2026/05/24	Redatta per la prima volta l'intera tabella dei Requisiti; piccoli raffinamenti nell'intero file	Sara Ristovic	Samuele Bertin
0.7.0	2026/05/21	Aggiunti 2 UC trasversali; raffinati gli UC dell'Agente OWASP	Sara Ristovic	Samuele Bertin
0.6.0	2026/05/19	Aggiunte estensioni 16 e 17 dell'UC 15; aggiornata la sezione "Requisiti"	Marco Barbiero	Dario Pirrone

Ver.	Data	Descrizione	Autore	Verificatore
0.5.0	2026/05/09	Scrittura della sottosezione "Casi d'uso trasversali"; Aggiunti 3 UC all'Agente OWASP, la sua introduzione e gli obiettivi e funzionalità	Sara Ristovic	Dario Pirrone
0.4.0	2026/04/30	Sviluppo sezioni "Casi d'uso" e Agenti	Sara Ristovic Marco Barbiero	Dario Pirrone
0.3.0	2026/04/15	Inizio scrittura della sezione "Requisiti"	Sara Ristovic	Marco Barbiero
0.2.0	2026/04/12	Scrittura della sezione 2	Dario Pirrone	Marco Barbiero
0.1.0	2026/04/08	Creazione documento e scrittura della sezione 1	Dario Pirrone	Marco Barbiero

Indice

1	Introduzione	6
1.1	Scopo del documento	6
1.2	Glossario	6
1.3	Riferimenti	6
1.3.1	Riferimenti normativi	6
1.3.2	Riferimenti informativi	6
2	Descrizione generale	7
2.1	Obiettivi del prodotto	7
2.2	Funzioni del prodotto	7
2.3	Caratteristiche degli utenti	8
2.4	Piattaforma di esecuzione	8
3	Casi d'Uso	9
3.1	Introduzione	9
3.2	Attori	9
3.2.1	Attori Primari	9
3.2.2	Attori Secondari	10
3.3	Casi d'Uso trasversali	11
3.3.1	UC 20: Registrazione di un nuovo utente	11
3.3.2	UC 21: Autenticazione dell'utente (Login)	12
3.3.3	UC 0: Configurazione delle credenziali	13
3.3.4	UC 1: Selezione <i>repository</i> e riferimento di base	14
3.3.5	UC 2: Selezione ambito di analisi	15
3.3.6	UC 3: Visualizzazione report prodotto dall'agente	16
3.3.7	UC 4: Validazione umana di una proposta dell'agente	17
3.3.8	UC 5: Visualizzazione errore durante l'esecuzione dell'agente	17
3.3.9	UC 6: Esportazione del report	18
4	Agente Docs: Documentazione e <i>README</i>	19
4.1	Introduzione all'agente	19
4.2	Obiettivi e Funzionalità	19
4.3	Casi d'Uso	20
4.3.1	UC 7: Gestione del <i>README</i> di Progetto	20
4.3.2	UC 8: Gestione della Documentazione Inline (<i>Docstring/JSDoc</i>)	21
4.3.3	UC 9: Gestione Documentazione <i>API</i>	22
5	Agente OWASP: Scansione <i>Vulnerabilità OWASP</i>	23
5.1	Introduzione all'agente	23
5.2	Obiettivi e Funzionalità	23
5.3	Casi d'Uso	24
5.3.1	UC 10: Esecuzione Scansione <i>Vulnerabilità (OWASP)</i>	24
5.3.2	UC 11: Verifica Conformità " <i>Policy-as-code</i> " (<i>CLAUDE.md</i>)	25
5.3.3	UC 12: Generazione Suggerimenti di <i>Remediation</i>	27
5.3.4	UC 13: Generazione snippet di codice correttivo	28

5.3.5	UC 14: Generazione descrizione testuale della <i>Remediation</i>	29
6	Agente Changelog: <i>Changelog</i> da <i>User Stories</i>	30
6.1	Introduzione all'agente	30
6.2	Obiettivi e Funzionalità	30
6.3	<i>Casi d'Uso</i>	31
6.3.1	UC 15: Recupero e Filtraggio <i>User Stories</i>	31
6.3.2	UC 16: Rilevamento di task con metadati insufficienti	32
6.3.3	UC 17: Gestione delle ambiguità e delle incoerenze	33
6.3.4	UC 18: Generazione <i>Changelog</i> Tecnico (Dev-to-Dev)	34
6.3.5	UC 19: Generazione <i>Changelog</i> di Business (Dev-to-Client)	35
7	Requisiti	37
7.1	Requisiti funzionali	37
7.2	Requisiti qualitativi	41
7.3	Requisiti di vincolo	42
7.4	Requisiti di sicurezza	42
8	Tracciamento	44
8.1	Tracciamento Fonte - Requisiti	44
8.2	Tracciamento Requisito - Fonti	45

Elenco delle tabelle

Elenco delle figure

1	Use Cases trasversali a tutti gli Agenti	11
2	Use Cases dell'Agente Docs	20
3	Use Case Scansione Vulnerabilità	24
4	Use Case Conformità "Policy-as-code"	25
5	Use Cases Generazione Suggerimenti di Remediation	27
6	Use Cases Recupero e Filtraggio User Stories	31
7	Use Case Generazione Changelog Tecnico	34
8	Use Case Generazione Changelog di Business	35

1 Introduzione

1.1 Scopo del documento

Il presente documento, denominato **Analisi dei Requisiti_G**, ha l'obiettivo di esporre in modo dettagliato e non ambiguo i requisiti funzionali, qualitativi e di vincolo individuati per il progetto **Code Guardian_G**. Tali requisiti sono stati delineati a seguito di un'attenta analisi del capitolato e dei successivi incontri con il **Proponente_G**. Il documento si avvale dell'uso di diagrammi e descrizioni dei *Casi d'Uso_G* (UC) per illustrare le interazioni tra gli *Attori_G* esterni e il sistema, fornendo una base solida per le successive fasi di progettazione, codifica e verifica.

1.2 Glossario

Per evitare ambiguità terminologiche e garantire una comprensione univoca del documento, i termini tecnici, gli acronimi e le parole che assumono un significato specifico nel contesto del progetto sono riportati nel documento *Glossario*. Questi termini sono contrassegnati nel presente documento da una "G" a pedice ed è un collegamento ipertestuale che rimanda direttamente alla definizione corrispondente nel *Glossario*. Tali parole vengono contrassegnate solamente alla loro prima occorrenza in questo documento.

1.3 Riferimenti

1.3.1 Riferimenti normativi

- **Norme di Progetto_G** v1.0.0
- **Piano di Qualifica_G** v1.0.0
- Capitolato d'appalto C2 - **Code Guardian**
- Regolamento del Progetto Didattico
- Verbale esterno 24 aprile
- Verbale esterno 23 marzo

1.3.2 Riferimenti informativi

- Glossario v1.0.0
- Dispense **Analisi dei Requisiti**
- Dispense *Casi d'Uso*

2 Descrizione generale

2.1 Obiettivi del prodotto

L'obiettivo principale del progetto è lo sviluppo di un set di agenti intelligenti basati su modelli linguistici di grandi dimensioni (*LLM_G*), progettati per integrarsi in un'architettura di orchestrazione già esistente fornita dal **Proponente Var Group**. Il sistema, denominato **Code Guardian**, mira ad automatizzare i processi di *Audit_G*, manutenzione e documentazione dei *Repository_G* software, riducendo il carico di lavoro manuale dei team di sviluppo e migliorando la qualità complessiva del codice. Nello specifico, il prodotto si prefigge di:

- **Garantire la coerenza documentale:** assicurando che il *README_G* di progetto e la documentazione tecnica (inline e *API_G*) siano costantemente allineati all'evoluzione funzionale della piattaforma.
- **Elevare gli standard di sicurezza:** introducendo il paradigma di *Policy-as-code_G*, attraverso il quale gli agenti verificano non solo le *Vulnerabilità_G* generiche *OWASP_G*, ma anche l'aderenza alle specifiche pratiche di sviluppo sicuro definite dal team.
- **Migliorare la trasparenza verso il business:** automatizzando la redazione di *Changelog_G* comprensibili al cliente finale, traducendo i progressi tecnici emersi dalle *User Stories_G* in valore di business tangibile.

2.2 Funzioni del prodotto

Il prodotto offre le sue funzionalità attraverso tre agenti specializzati coordinati dall'**Orchestratore_G** centrale:

1. Agente di Documentazione e *README*:

- **Manutenzione *README*:** creazione e aggiornamento dinamico del file *README* di progetto in base alle modifiche apportate alla piattaforma.
- **Gestione documentazione tecnica:** generazione e aggiornamento di documentazione inline (*Docstring_G*, *JSDoc_G*) e delle *API docs_G* per riflettere lo stato attuale del codice.

2. Agente di Security Review (*Policy-as-code*):

- **Analisi *Vulnerabilità*:** utilizzo delle *API* di *Claude Code_G* per l'identificazione di *Vulnerabilità* standard *OWASP*.
- **Verifica policy interne:** controllo dell'adozione di pratiche di sviluppo sicuro specifiche del team, utilizzando i file di configurazione *CLAUDE.md* presenti nel *Repository* come contesto operativo.

3. Agente di *Changelog* da *User Stories*:

- **Analisi *Sprint_G***: lettura e filtraggio delle *User Stories* completate durante uno specifico *Sprint* di sviluppo.
- **Traduzione Business-Oriented**: conversione del linguaggio tecnico delle task in un linguaggio chiaro e comprensibile per il cliente, producendo *Changelog* sia tecnici che di business.

2.3 Caratteristiche degli utenti

Il sistema è progettato per inserirsi in un flusso di lavoro di sviluppo software moderno, interagendo con diverse figure professionali che operano all'interno di un team IT o nel rapporto con il cliente finale.

L'utente principale è rappresentato dallo **sviluppatore**, il quale interagisce con il sistema indirettamente attraverso il *Repository *Git_G*. Egli beneficia dell'automatizzazione della documentazione tecnica e del feedback immediato sulla sicurezza, che gli permette di correggere eventuali *Vulnerabilità* o difformità rispetto alle policy aziendali in tempo reale, senza attendere revisioni manuali.*

Una seconda figura chiave è il *Team Lead_G* o lo *Security Auditor_G*. Questi utenti utilizzano il sistema come strumento di monitoraggio e governance: definiscono le regole di "sviluppo sicuro" all'interno dei file CLAUDE.md e delegano all'agente di sicurezza il compito di verificarne l'adozione costante su tutto il codice prodotto.

Infine, il sistema si rivolge agli *Stakeholder_G* di business (come *Project Manager_G* o il cliente finale). Questi soggetti non necessitano di competenze tecniche approfondite, ma hanno bisogno di visibilità sull'andamento del progetto. Per loro, l'agente di *Changelog* funge da "traduttore", trasformando il gergo tecnico delle *User Stories* in report di valore che sintetizzano i progressi funzionali raggiunti al termine di ogni *Sprint*.

2.4 Piattaforma di esecuzione

Il prodotto è concepito come un ecosistema di agenti integrati in un'infrastruttura già esistente, operante principalmente in ambiente cloud *AWS_G*. Essendo il sistema basato su interfacce di programmazione e automazione, l'esecuzione è garantita attraverso i principali ambienti di runtime come *Node.js_G* e *Python_G*. L'interazione con il codice sorgente avviene tramite repository Git. L'eventuale integrazione con *GitHub Actions_G* potrà essere utilizzata per automatizzare l'esecuzione degli agenti in seguito a eventi come push o *Pull Request_G*, qualora tale funzionalità venga inclusa nel perimetro del *MVP_G*.

3 *Casi d'Uso*

3.1 Introduzione

I *Casi d'Uso* sono costituiti da un diagramma *UML_G* e da una descrizione testuale che ne chiarisce il comportamento e gli obiettivi, al fine di definire in modo preciso le funzionalità che il prodotto deve offrire.

3.2 *Attori*

Il sistema **Code Guardian** interagisce con diverse entità fisiche e logiche per portare a termine le proprie funzionalità. Al fine di descrivere in modo chiaro e non ambiguo i *Casi d'Uso*, gli *Attori* sono stati classificati in due categorie: **Attori Primari**, ovvero gli utenti umani che interagiscono direttamente o indirettamente con il sistema per raggiungere un obiettivo, e **Attori Secondari**, ovvero i sistemi, le piattaforme o i file esterni chiamati in causa dall'applicazione per completare le operazioni richieste.

Gli *Attori* primari sono strutturati secondo una relazione gerarchica di generalizzazione e specializzazione: l'*Attore* di base definisce le caratteristiche e le facoltà comuni che vengono interamente ereditate e successivamente estese dalle figure specializzate.

3.2.1 *Attori Primari*

- **Utente non Autenticato:**
- **Utente Generico:** Rappresenta l'*Attore* di base del sistema dal quale ereditano tutti gli altri *Attori* primari. Questo ruolo racchiude le facoltà e le interazioni comuni a qualsiasi utilizzatore della piattaforma, consentendo l'esecuzione dei casi d'uso definiti come trasversali, tra cui la selezione del *Repository Git*, la visualizzazione dei messaggi di errore e l'esportazione dei report prodotti.
- **Sviluppatore:** Specializzazione di Utente Generico ed utente principale del sistema. Interagisce con gli agenti per delegare la redazione e l'aggiornamento della documentazione tecnica, per validare le proposte di modifica (approccio *Human-in-the-loop_G*) e per ricevere feedback immediati sulle vulnerabilità di sicurezza del codice prodotto.
- **Security Auditor / Team Lead:** Specializzazione di Utente Generico con responsabilità di governance. Definisce le regole di sviluppo sicuro e le policy interne tramite il file di configurazione `CLAUDE.md`, che l'Agente *OWASP* utilizzerà per verificare la conformità del codice prodotto dal team secondo il paradigma *Policy-as-code*.
- **Project Manager:** Specializzazione di Utente Generico non necessariamente tecnica che necessita di visibilità sull'andamento del progetto. Interagisce con l'Agente *Changelog* innescando la generazione dei report a conclusione di uno *Sprint*, al fine di ottenere una traduzione orientata al business delle *User Stories* completate.

3.2.2 Attori Secondari

- **Repository Git:** Il sistema di controllo di versione esterno (*GitHub_G*). Rappresenta sia la fonte dei dati su cui gli agenti effettuano le analisi, sia la destinazione delle proposte di modifica tramite *Pull Request* o *Patch_G*.
- **API Claude Code_G / Modello AI (LLM):** I servizi di intelligenza artificiale di grandi dimensioni (*LLM*) interrogati dagli agenti per individuare vulnerabilità complesse, generare codice correttivo e produrre documentazione tecnica descrittiva.
- **Sistema di Task Management:** La piattaforma esterna di tracciamento delle attività. Viene interrogata dall'Agente *Changelog* per recuperare i metadati, le descrizioni e i criteri di accettazione dei *Ticket_G* completati durante uno *Sprint*.

3.3 *Casi d'Uso* trasversali

Di seguito sono riportati i *Casi d'Uso* trasversali, ovvero validi per due o più agenti, mentre quelli specifici verranno esposti nelle sotto-sezioni apposite all'interno di ciascuna sezione dedicata a ciascun agente.

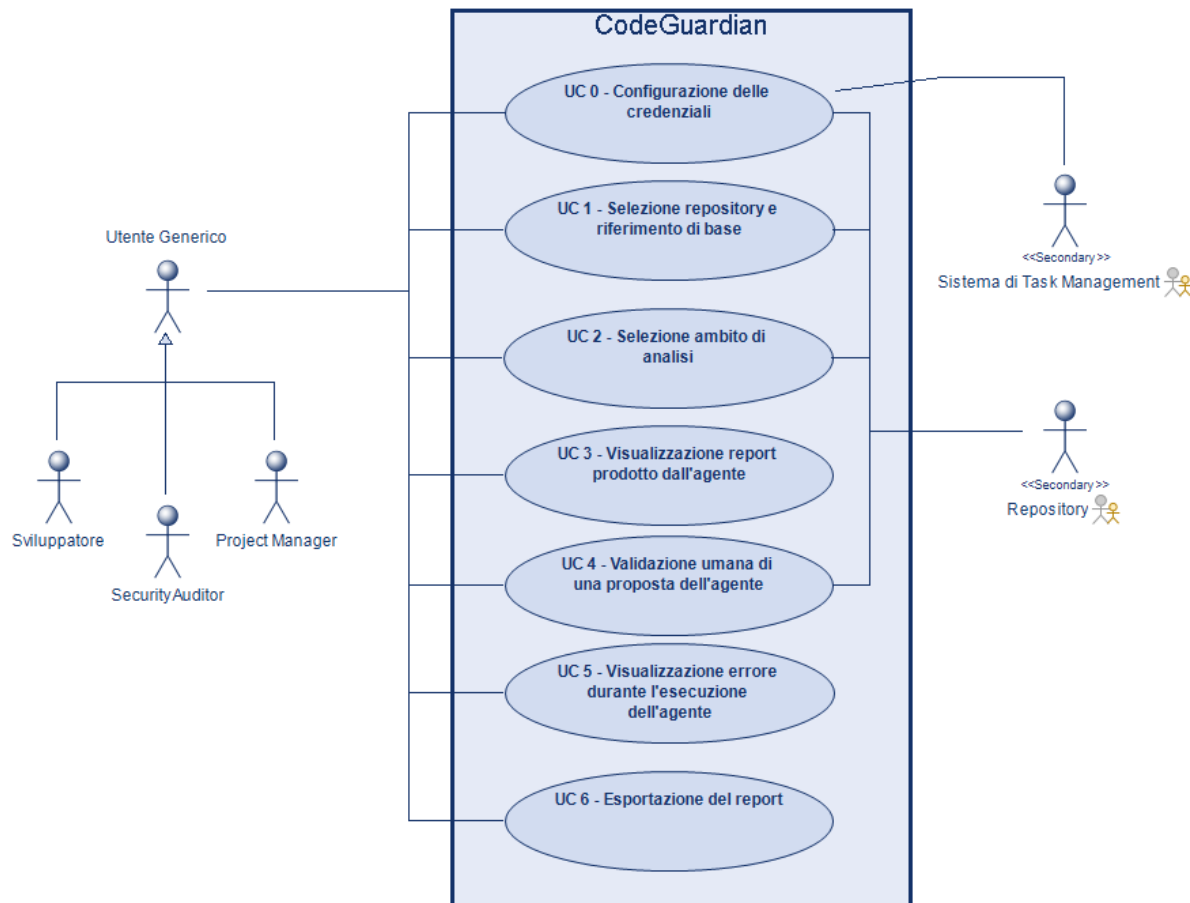


Figura 1: Use Cases trasversali a tutti gli Agenti

3.3.1 UC 20: Registrazione di un nuovo utente

Questo caso d'uso descrive il processo attraverso il quale un utente non ancora registrato (non riconosciuto) crea un nuovo account all'interno del sistema **Code Guardian**, definendo contestualmente il proprio ruolo operativo per l'accesso alle funzionalità degli agenti.

- **Attore Primario:** Utente non Autenticato.
- **Trigger:** L'utente accede per la prima volta alla piattaforma **Code Guardian** e desidera creare un account.
- **Precondizioni:**
 - Il sistema è attivo e accessibile.

- L'utente non possiede ancora un account registrato associato al proprio indirizzo email.

- **Postcondizioni:**

- L'account dell'utente è stato creato con successo nel sistema.
- All'utente è stato associato il ruolo selezionato (*Sviluppatore*, *Security Auditor* o *Project Manager*).

- **Scenario Principale:**

1. L'utente accede alla pagina di registrazione del sistema **Code Guardian**.
2. Il sistema richiede l'inserimento dei dati anagrafici e delle credenziali di accesso (Nome, Cognome, Indirizzo Email, Password).
3. Il sistema richiede la selezione del ruolo operativo che l'utente assumerà all'interno del progetto, offrendo come opzioni: *Sviluppatore*, *Security Auditor*, *Project Manager*.
4. L'utente compila i campi richiesti e seleziona il proprio ruolo.
5. Il sistema convalida la correttezza formale dei dati inseriti (es. validità dell'email, robustezza della password).
6. Il sistema registra il nuovo utente, memorizza le credenziali in modo sicuro e notifica il successo dell'operazione.

- **Scenari alternativi:**

- **Email già registrata:** Al passo 5, se il sistema rileva che l'indirizzo email inserito è già associato a un account esistente, l'operazione viene interrotta. Il sistema mostra un messaggio di errore e suggerisce di procedere con il login (UC 21).
- **Dati non validi:** Al passo 5, se i dati inseriti non rispettano i criteri di validazione (password troppo debole o formato email errato), il sistema mostra un messaggio di errore specifico e chiede all'utente di correggere i campi evidenziati.

3.3.2 UC 21: Autenticazione dell'utente (Login)

Questo caso d'uso descrive il processo tramite il quale un utente già registrato, ma non ancora riconosciuto nella sessione corrente, fornisce le proprie credenziali per accedere al sistema e ottenere i permessi associati al proprio ruolo.

- **Attore Primario:** Utente non autenticato.
- **Trigger:** L'utente registrato desidera accedere al sistema per utilizzare gli agenti o visualizzare i report di Code Guardian.

- **Precondizioni:**

- L'utente ha completato con successo il processo di registrazione (UC 20).
- Il sistema è attivo e accessibile.

- **Postcondizioni:**

- L'utente è autenticato con successo.
- Il sistema riconosce l'utente identificandolo come Utente Generico specializzato nel ruolo scelto in fase di registrazione (Sviluppatore, Security Auditor o Project Manager), sbloccando i relativi permessi e funzionalità.

- **Scenario Principale:**

1. L'utente accede alla pagina di login del sistema.
2. Il sistema richiede l'inserimento delle credenziali di accesso (Indirizzo Email e Password).
3. L'utente inserisce le proprie credenziali.
4. Il sistema verifica la corrispondenza delle credenziali inserite con quelle memorizzate in modo sicuro nel database.
5. Il sistema autentica l'utente, avvia una sessione attiva associata al suo ruolo e lo reindirizza alla dashboard principale.

- **Scenari alternativi:**

- **Credenziali errate:** Al passo 4, se l'indirizzo email o la password non trovano corrispondenza nel sistema, l'operazione viene interrotta. Il sistema mostra un messaggio di errore generico (es. "Credenziali non valide") e invita l'utente a riprovare.

3.3.3 UC 0: Configurazione delle credenziali

Questo caso d'uso descrive il processo attraverso il quale l'utente fornisce al sistema le credenziali necessarie per autorizzare l'accesso ai servizi esterni (come il *Repository Git* e il Sistema di Task Management).

- **Attore Primario:** Utente Generico.
- **Attori Secondari:** *Repository Git*, Sistema di Task Management.
- **Trigger:** L'utente accede per la prima volta al sistema o necessita di aggiornare un token scaduto.
- **Precondizioni:**
 - Il sistema è attivo e accessibile.

- **Postcondizioni:** I token di accesso sono memorizzati in modo sicuro nel sistema e gli agenti sono pronti all'esecuzione.
- **Scenario Principale:**
 1. L'utente accede alla sezione di configurazione del sistema.
 2. Il sistema richiede l'inserimento dei parametri di autenticazione.
 3. L'utente inserisce i dati richiesti.
 4. Il sistema convalida la correttezza formale e effettua una chiamata di test per verificare i permessi delle chiavi inserite.
 5. Il sistema salva le credenziali e notifica all'utente il successo dell'operazione.
- **Scenari alternativi:**
 - Al passo 4, se la chiamata di test fallisce, il sistema mostra un messaggio di errore all'utente chiedendo di reinserire le credenziali.

3.3.4 UC 1: Selezione *repository* e riferimento di base

Questo caso d'uso consente all'utente di indicare il repository Git e il riferimento di base su cui potranno essere eseguite le successive operazioni degli agenti. La selezione del repository non implica necessariamente l'analisi dell'intero contenuto del repository, ma definisce il contesto iniziale da utilizzare nelle funzionalità successive. È una fase preliminare comune alle funzionalità di analisi, documentazione, sicurezza e generazione di report.

- **Attore Primario:** Utente Generico.
- **Attori Secondari:** *repository* Git.
- **Trigger:** L'utente vuole selezionare un *repository* e un riferimento specifico da utilizzare come contesto per gli agenti disponibili.
- **Precondizioni:**
 - Il sistema è attivo e accessibile.
 - Il *repository* Git esiste ed è raggiungibile.
 - L'utente dispone dei permessi necessari per accedere al *repository*.
 - Il repository selezionato contiene codice sorgente scritto nei linguaggi supportati dal sistema.
- **Postcondizioni:** Il *repository* e il riferimento selezionati sono disponibili come contesto per le funzionalità degli agenti.
- **Scenario Principale:**

1. L'utente accede alla sezione dedicata alla selezione del *repository*.
2. L'utente fornisce l'*URL* del *repository* Git.
3. Il sistema verifica che il *repository* sia raggiungibile e accessibile.
4. L'utente seleziona il riferimento di base da utilizzare, scegliendo tra branch, *Commit_G* o *Pull Request*.
5. Se l'utente seleziona un branch senza indicare un *Commit* specifico, il sistema considera di default l'ultimo *Commit* disponibile sul branch selezionato.
6. Il sistema registra il *repository* e il riferimento selezionati come contesto per le successive operazioni degli agenti.

- **Scenari alternativi:**

- Se l'*URL* del *repository* non è valido, il sistema mostra un messaggio di errore esplicativo.
- Se il *repository* non è raggiungibile o l'utente non dispone dei permessi necessari, il sistema mostra un messaggio di errore esplicativo.
- Se il branch, il *Commit* o la *Pull Request* selezionati non esistono, il sistema mostra un messaggio di errore esplicativo.
- Se il repository contiene un linguaggio non supportato, il sistema avvisa l'utente che le funzionalità di analisi potrebbero non essere disponibili o generare errori.

3.3.5 UC 2: Selezione ambito di analisi

Questo caso d'uso consente all'utente di delimitare la porzione del repository su cui eseguire l'analisi di un agente. L'ambito può essere ristretto a uno o più file o directory. L'analisi dell'intero repository è considerata una possibilità aggiuntiva solo se supportata dalla configurazione del sistema.

- **Attore Primario:** Utente Generico.
- **Attori Secondari:** Repository Git.
- **Trigger:** L'utente vuole delimitare la parte del repository da sottoporre all'analisi di un agente.
- **Precondizioni:**
 - È stato selezionato un repository e un riferimento di base (UC 1).
 - Il sistema può accedere al contenuto del repository selezionato.
- **Postcondizioni:** L'ambito di analisi selezionato è disponibile come input per l'agente interessato.

- **Scenario Principale:**

1. L'utente seleziona l'ambito dell'analisi, indicando o l'intero *Repository* o porzioni minori (singoli file o directory).
2. Il sistema verifica che gli elementi indicati esistano e siano accessibili.
3. Il sistema registra l'ambito selezionato come input per l'esecuzione dell'agente interessato.

- **Scenari alternativi:**

- Se l'utente non seleziona alcun ambito, il sistema mostra un messaggio informativo e richiede di completare la selezione.
- Se uno o più elementi selezionati non esistono o non sono accessibili, il sistema mostra un messaggio di errore esplicativo.
- (desiderabile) Se l'ambito selezionato supera la capacità di elaborazione del modello, il sistema mostra un messaggio di errore esplicativo e richiede di restringere la selezione.

3.3.6 UC 3: Visualizzazione report prodotto dall'agente

L'utente vuole consultare il risultato prodotto da un agente a seguito di un'analisi. Il sistema rende disponibili i report generati, permettendo all'utente di visualizzare le informazioni principali relative all'esecuzione, come criticità rilevate, suggerimenti proposti e file coinvolti.

- **Attore Primario:** Utente Generico.

- **Trigger:** L'utente vuole consultare l'esito dell'analisi svolta da un agente.

- **Precondizioni:**

- L'**Orchestratore** ha eseguito almeno un agente ed è disponibile almeno un report prodotto dall'agente.

- **Postcondizioni:** L'utente visualizza il report generato.

- **Scenario Principale:**

1. L'utente accede alla sezione dedicata ai risultati dell'analisi.
2. Il sistema mostra la lista dei report disponibili.
3. L'utente seleziona il report desiderato.
4. Il sistema visualizza il contenuto del report, includendo eventuali criticità, suggerimenti e file coinvolti.

- **Scenari alternativi:**

- Se non sono presenti report, il sistema mostra un messaggio informativo.

3.3.7 UC 4: Validazione umana di una proposta dell'agente

Il sistema adotta un approccio *Human-in-the-loop*, in modo che le proposte generate dagli agenti siano validate da un utente prima di essere applicate.

- **Attore Primario:** Utente Generico.
- **Attori Secondari:** *Repository Git*.
- **Trigger:** L'utente riceve una proposta generata da un agente.
- **Precondizioni:**
 - Un agente ha prodotto una proposta di modifica. La proposta è disponibile sotto forma di *Pull Request*, frammento di codice o suggerimento testuale.
- **Postcondizioni:** La proposta è stata accettata, modificata o rifiutata dall'utente.
- **Scenario Principale:**
 1. L'utente apre la proposta generata dall'agente.
 2. L'utente analizza il contenuto della proposta.
 3. L'utente decide se accettare, modificare o rifiutare la proposta.
 4. Il sistema registra l'esito della validazione.

3.3.8 UC 5: Visualizzazione errore durante l'esecuzione dell'agente

Durante l'esecuzione di un agente può verificarsi un errore che impedisce il completamento dell'analisi o la generazione dell'output richiesto. In questo caso il sistema mostra all'utente un messaggio esplicativo, indicando la causa dell'errore e, quando possibile, una possibile azione correttiva.

- **Attore Primario:** Utente Generico.
- **Trigger:** L'esecuzione di un agente termina con errore.
- **Precondizioni:**
 - È stata avviata l'esecuzione di un agente. Si verifica un errore durante l'analisi o la generazione dell'output.
- **Postcondizioni:** L'utente visualizza un messaggio di errore esplicativo.
- **Scenario Principale:**
 1. L'agente incontra un errore durante l'esecuzione.
 2. L'**Orchestratore** riceve l'esito negativo.

3. Il sistema mostra all'utente un messaggio che descrive la causa dell'errore (es. timeout di risposta dal modello *AI*, formato di output dell'*LLM* non interpretabile, file di configurazione assente, formato del report non generabile, superamento del limite massimo di file/righe di codice elaborabili, superamento delle quote di utilizzo API).
4. Il sistema indica, se possibile, un'azione correttiva.

3.3.9 UC 6: Esportazione del report

L'utente vuole esportare un report generato da un agente per conservarlo, condividerlo o utilizzarlo nelle successive attività di sviluppo e verifica. Il sistema permette di selezionare il report desiderato e di salvarlo in uno dei formati disponibili.

- **Attore Primario:** Utente Generico.
- **Trigger:** L'utente vuole esportare e salvare il risultato di un'analisi di un agente in uno dei formati disponibili.
- **Precondizioni:**
 - È disponibile almeno un report generato da un agente.
- **Postcondizioni:** Il report è stato esportato nel formato scelto.
- **Scenario Principale:**
 1. L'utente seleziona un report.
 2. L'utente sceglie il formato di esportazione.
 3. Il sistema genera il file esportabile.
 4. L'utente scarica o salva il report.
- **Scenari alternativi:**
 - Se l'esportazione fallisce, il sistema mostra un messaggio di errore.

4 Agente Docs: Documentazione e *README*

4.1 Introduzione all'agente

L'Agente di Documentazione e *README* rappresenta un pilastro fondamentale per la qualità del lavoro quotidiano degli sviluppatori e per la stabilità del progetto Code Guardian. Sappiamo bene come, nei cicli di sviluppo veloci, la documentazione sia spesso la prima a perdere il passo rispetto al codice, diventando presto superata. Questo agente nasce proprio per risolvere tale problema: il suo compito è automatizzare tutto il processo di aggiornamento. In questo modo, garantiamo che il file *README.md* e l'intera documentazione tecnica, sia quella interna al codice che quella dedicata alle *API*, siano sempre uno specchio fedele e puntuale dell'evoluzione del progetto, in qualsiasi momento.

4.2 Obiettivi e Funzionalità

L'agente si focalizza su tre aree critiche:

1. **Manutenzione del *README*:** Automazione della creazione e dell'aggiornamento del file principale di presentazione del *Repository*, adattandolo dinamicamente alle evoluzioni della piattaforma.
2. **Gestione della documentazione inline:** Analisi semantica del codice per la generazione di *Docstring* e *JSDoc*, assicurando che ogni modulo sia correttamente commentato.
3. **Documentazione delle *API*:** Estrazione automatizzata degli *Endpoint_G* per la creazione di guide tecniche per gli utilizzatori del servizio.

4.3 Casi d'Uso

Di seguito i *Casi d'Uso* dell'Agente Docs.

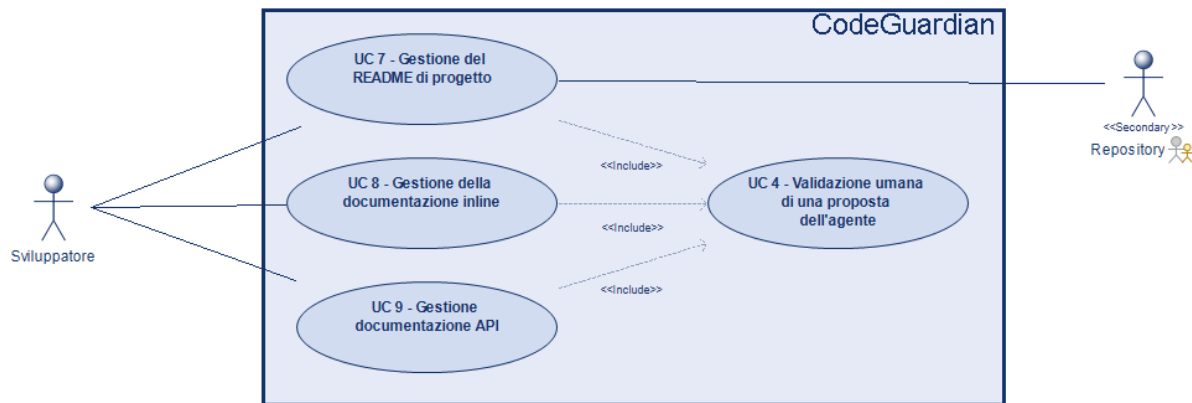


Figura 2: Use Cases dell'Agente Docs

4.3.1 UC 7: Gestione del *README* di Progetto

Questo è il *Caso d'Uso* generico per la gestione automatizzata del *README.md* del Progetto.

- **Attore Primario:** Sviluppatore.
- **Attori Secondari:** *Repository Git*.
- **Trigger:** Lo Sviluppatore richiede l'intervento tramite l'**Orchestratore**.
- **Precondizioni:**
 - È stato selezionato un *repository* e un riferimento di base (UC 1).
 - (Opzionale) Esiste un modello personalizzato per il *README*.
- **Postcondizioni:** È disponibile una proposta di creazione o aggiornamento del *README.md*, pronta per la validazione umana, o il *README.md* è già aggiornato.
- **Scenario Principale:**
 1. L'**Orchestratore** notifica l'Agente Docs della necessità di gestire il *README*.
 2. L'Agente analizza file di configurazione e struttura delle cartelle.
 3. L'Agente crea o aggiorna il *README* secondo il modello personalizzato, se disponibile, o quello di default.
 4. L'Agente propone il prodotto tramite *Pull Request*.
- **Scenario alternativo:**

- L’analisi non rileva cambiamenti: non vengono generate proposte di aggiornamento, lo Sviluppatore viene informato e lo UC termina.
 - L’Agente non riceve risposta dall’*LLM* entro il tempo limite oppure riceve un output la cui struttura non è parsabile dal sistema, l’elaborazione viene interrotta e il flusso passa all’UC 5 per la visualizzazione dell’errore.
- **Inclusioni:** UC 4 - Validazione umana di una proposta dell’agente.

4.3.2 UC 8: Gestione della Documentazione Inline (*Docstring/JSDoc*)

L’agente interviene nel codice sorgente per garantire che ogni componente sia documentato correttamente.

- **Attore Primario:** Sviluppatore.
- **Trigger:** Richiesta esplicita da parte dello Sviluppatore verso l’Orchestratore.
- **Precondizioni:**
 - È stato selezionato un ambito di analisi (UC 2).
- **Postcondizioni:** Lo Sviluppatore dispone di una proposta di documentazione inline da validare o la documentazione è già aggiornata.
- **Scenario Principale:**
 1. L’Agente individua funzioni o classi prive di documentazione, con documentazione obsoleta o non conforme.
 2. L’Agente interpreta la logica della funzione, i parametri di input e il valore di ritorno o lo scopo e le caratteristiche della classe.
 3. Per ogni classe/funzione individuata, l’Agente genera una proposta di commento standardizzato (*JSDoc*).
 4. Il sistema rende le proposte disponibili per validazione allo Sviluppatore.
- **Scenario alternativo:**
 - Se l’Agente non riesce a interpretare una classe/funzione, invece di una proposta viene generato un avvertimento per quella specifica classe/funzione e l’analisi passa a quella seguente, se presente.
 - Se le modifiche al codice non richiedono di cambiare la documentazione, non vengono generate proposte di aggiornamento, lo Sviluppatore viene informato e l’UC termina.
 - Se l’Agente non riceve risposta dall’*LLM* entro il tempo limite oppure riceve un output la cui struttura non è parsabile dal sistema, l’elaborazione viene interrotta e il flusso passa all’UC 5 per la visualizzazione dell’errore.

- **Inclusioni:**

- UC 4: Validazione umana di una proposta dell'agente.

4.3.3 UC 9: Gestione Documentazione *API*

L'agente estrae le definizioni degli *Endpoint* per creare una guida tecnica per i consumatori delle *API*.

- **Attore Primario:** Sviluppatore.

- **Trigger:** Richiesta esplicita di generazione prima di un rilascio.

- **Precondizioni:**

- È stato selezionato un ambito di analisi (UC 2).

- **Postcondizioni:** È disponibile una proposta di documento tecnico (*API.md*) o il documento è già aggiornato.

- **Scenario Principale:**

1. L'Agente scansiona i file dei controller e i file delle *Rotte_G*.
2. Estrazione di: metodo HTTP, URL, header, *Payload_G* richiesto e risposte possibili.
3. Formattazione delle informazioni nel file *API.md* in formato *Markdown_G*.
4. Il Sistema mette a disposizione la proposta di documento allo Sviluppatore per la validazione.

- **Scenario alternativo:**

- Se l'analisi non rileva cambiamenti, non vengono generate proposte di aggiornamento, lo Sviluppatore è informato e l'UC termina.
- Se l'Agente non riceve risposta dall'*LLM* entro il tempo limite oppure riceve un output la cui struttura non è parsabile dal sistema, l'elaborazione viene interrotta e il flusso passa all'UC 5 per la visualizzazione dell'errore.

- **Inclusioni:**

- UC 4: Validazione umana di una proposta dell'agente.

5 Agente OWASP: Scansione *Vulnerabilità OWASP*

5.1 Introduzione all'agente

L'Agente di Scansione Vulnerabilità *OWASP* ha l'obiettivo di supportare il *Security Auditor* nell'individuazione di possibili *Vulnerabilità* di sicurezza presenti nel codice sorgente del *Repository*. L'analisi di sicurezza viene svolta facendo riferimento alle principali categorie di rischio definite da *OWASP* individuabili tramite analisi del codice, come: *Injection_G*, gestione non sicura dell'autenticazione, controllo degli accessi incompleto, gestione errata di dati sensibili, configurazioni insicure, ecc.

L'agente non garantisce la copertura esaustiva dell'intera *OWASP* Top 10. Il suo scopo è fornire una revisione automatizzata assistita, producendo segnalazioni, priorità e suggerimenti di *Remediation_G* che devono essere successivamente valutati da un *Team Lead* o *Security Auditor*.

5.2 Obiettivi e Funzionalità

L'agente si concentra su tre aree principali:

1. **Analisi delle *Vulnerabilità OWASP*:** individuazione di pattern di codice potenzialmente riconducibili a *Vulnerabilità* note, come *SQL Injection*, uso insicuro di funzioni crittografiche, esposizione di informazioni sensibili, *Hardcoded secrets_G*, ecc.
2. **Verifica delle policy interne di sicurezza:** controllo dell'aderenza del codice alle regole definite dal team nel file *CLAUDE.md*, secondo un approccio di tipo *Policy-as-code*.
3. **Generazione di remediation:** produzione di suggerimenti testuali o modifiche al codice correttive per mitigare le vulnerabilità o le non conformità rilevate, senza applicare modifiche automatiche prima della validazione umana.

5.3 *Casi d'Uso*

Di seguito i *Casi d'Uso* dell'Agente OWASP.

5.3.1 UC 10: Esecuzione Scansione *Vulnerabilità (OWASP)*

L'agente utilizza le *API* di *Claude Code* per identificare *Vulnerabilità* standard di sicurezza nel codice sorgente.

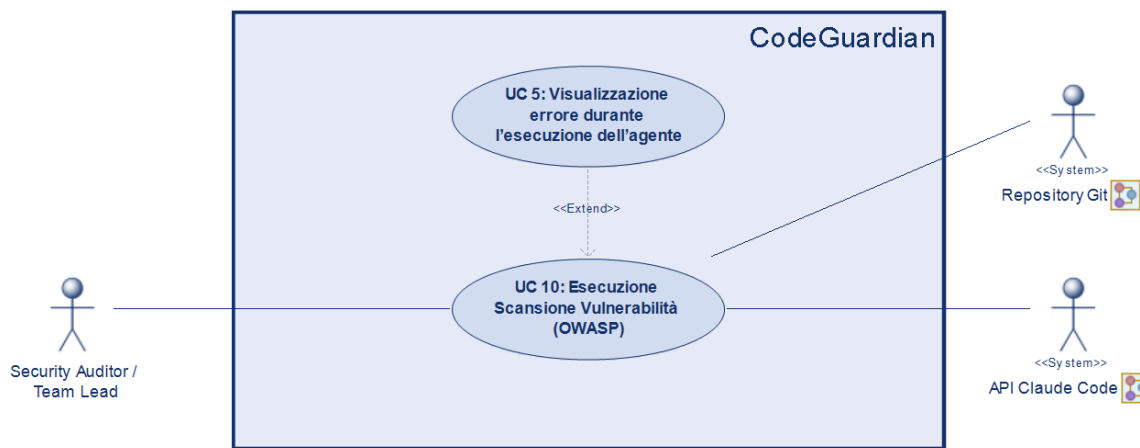


Figura 3: Use Case Scansione Vulnerabilità

- **Attore Primario:** *Security Auditor / Team Lead*.
- **Attori Secondari:** *Repository Git*, *API Claude Code*.
- **Trigger:** Il *Security Auditor* richiede una scansione di sicurezza per identificare *Vulnerabilità* all'**Orchestratore**.
- **Precondizioni:**
 - È stato selezionato un ambito di analisi (UC 2).
 - L'Agente OWASP è correttamente configurato con le *API Key* di *Claude Code*.
- **Postcondizioni:** Viene generato un report tecnico delle *Vulnerabilità* rilevate. Ogni *Vulnerabilità* rilevata è descritta con categoria, gravità e riferimento al codice interessato.
- **Scenario Principale:**
 1. L'Agente OWASP riceve dall'**Orchestratore** il codice sorgente relativo all'ambito di analisi selezionato.
 2. L'Agente OWASP interroga le *API* di *Claude Code* inviando il codice per l'analisi.

3. L'Agente OWASP analizza il codice cercando pattern riconducibili alle *Vulnerabilità* della Top 10 OWASP (es. *SQL Injection*, *XSS_G*, *Hardcoded secrets*).
4. L'Agente OWASP riceve l'esito dell'analisi.
5. L'Agente OWASP associa ogni *Vulnerabilità* al file e, se disponibile, alla porzione di codice interessata.
6. L'Agente OWASP classifica ogni *Vulnerabilità* trovata nelle categorie di Top 10 OWASP.
7. L'Agente OWASP formatta i risultati evidenziando la gravità (High, Medium, Low) di ogni falla trovata.
8. L'Agente OWASP inserisce le *Vulnerabilità*, la loro classificazione e le gravità nel report finale.

• **Scenario Alternativo:**

- Se non vengono rilevate *Vulnerabilità*, l'Agente genera un report che indica l'assenza di criticità individuate.
- L'Agente non riceve risposta dall'*LLM* entro il tempo limite oppure riceve un output la cui struttura non è parsabile dal sistema, l'elaborazione viene interrotta e il flusso passa all'UC 5 per la visualizzazione dell'errore.

5.3.2 UC 11: Verifica Conformità "*Policy-as-code*" (CLAUDE.md)

L'agente verifica che il codice rispetti le regole specifiche definite dal team nel file di configurazione.

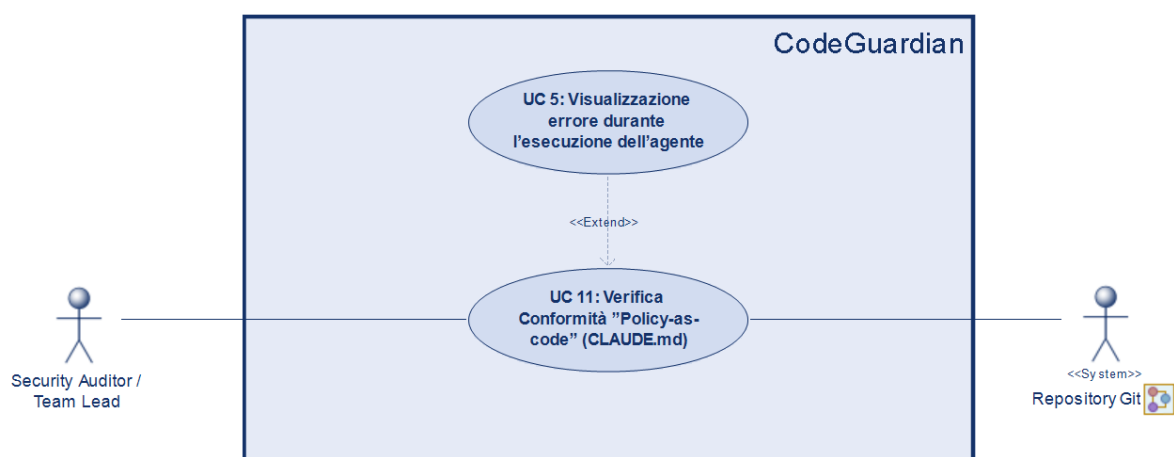


Figura 4: Use Case Conformità "*Policy-as-code*"

- **Attore Primario:** *Security Auditor / Team Lead*.

- **Attori Secondari:** *Repository Git*.
- **Trigger:** Il *Security Auditor* richiede una revisione di conformità all'**Orchestratore**.
- **Precondizioni:**
 - È stato selezionato un ambito di analisi (UC 2).
 - Il codice sorgente è accessibile.
- **Postcondizioni:** Viene prodotto un elenco di non-conformità rispetto alle policy di progetto.
- **Scenario Principale:**
 1. L'Agente verifica che il file **CLAUDE.md** sia presente, leggibile e interpretabile.
 2. L'Agente OWASP legge il contenuto del file **CLAUDE.md** per estrarre il contesto e le regole (es. "usa sempre questa libreria per il logging", "non usare variabili globali").
 3. L'Agente analizza il codice sorgente confrontandolo con le direttive estratte.
 4. L'Agente identifica eventuali deviazioni dalle pratiche concordate dal team.
 5. L'Agente genera un elenco di violazioni della policy interna.
- **Scenari alternativi:**
 - **Il file **CLAUDE.md** non è presente:** il sistema rileva l'assenza del file, interrompe la verifica delle policy interne e delega la gestione del fallimento all'UC 5 per mostrare il messaggio d'errore specifico.
 - **Il file **CLAUDE.md** non è leggibile o non contiene regole interpretabili:** il sistema rileva l'impossibilità di analizzare il file, interrompe la verifica delle policy interne e delega la gestione del fallimento all'UC 5 per mostrare il messaggio d'errore specifico.
 - **Il file **CLAUDE.md** contiene regole in palese contraddizione logica:** l'Agente OWASP legge il contenuto del file per estrarre le regole, rileva un conflitto logico insormontabile tra due o più direttive, quindi interrompe la verifica delle policy interne e delega la gestione del fallimento all'UC 5.

5.3.3 UC 12: Generazione Suggerimenti di *Remediation*

L'agente fornisce soluzioni pratiche per correggere i problemi di sicurezza o le violazioni di policy trovate.

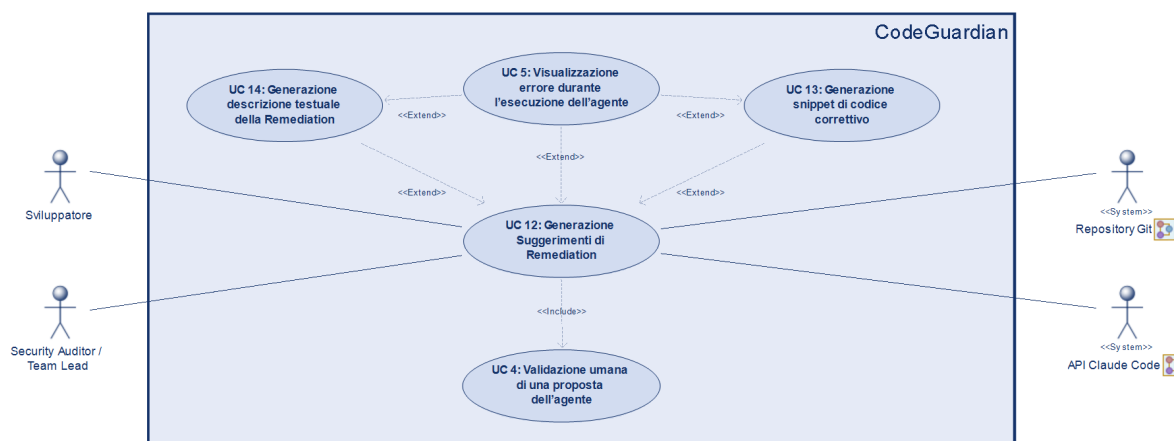


Figura 5: Use Cases Generazione Suggerimenti di Remediation

- **Attore Primario:** *Security Auditor / Team Lead / Sviluppatore.*
- **Attori Secondari:** *Repository Git, API Claude Code.*
- **Trigger:** Conclusione con esito positivo (rilevamento falle) degli UC 10 o UC 11 e richiesta del utente di generare suggerimenti di *Remediation*.
- **Precondizioni:**
 - Sono state identificate *Vulnerabilità* o violazioni di policy.
- **Postcondizioni:** Il *Security Auditor* riceve indicazioni chiare su come correggere il codice per renderlo conforme.
- **Scenario Principale:**
 1. Per ogni vulnerabilità o violazione rilevata, l'Agente analizza il contesto e la porzione di codice interessata.
 2. L'Agente determina il livello di localizzazione possibile per la risoluzione del problema.
 3. Il flusso si estende per generare uno snippet di codice correttivo (UC 13) oppure una descrizione testuale (UC 14).
 4. Il *Security Auditor* consulta vulnerabilità, gravità, file coinvolto, spiegazione e proposta correttiva.
- **Scenario Alternativo:**

- L'Agente non riceve risposta dall'*LLM* entro il tempo limite oppure riceve un output la cui struttura non è parsabile dal sistema, l'elaborazione viene interrotta e il flusso passa all'UC 5 per la visualizzazione dell'errore.

- **Inclusioni:**

- UC 4: Validazione umana di una proposta dell'agente.

- **Estensioni:**

- UC 13: Generazione snippet di codice correttivo.
- UC 14: Generazione descrizione testuale della remediation.

5.3.4 UC 13: Generazione snippet di codice correttivo

Descrive la generazione di un frammento di codice alternativo per correggere una *Vulnerabilità* o una non conformità.

- **Attore Primario:** *Security Auditor / Team Lead / Sviluppatore*.
- **Attori Secondari:** *Repository Git, API Claude Code*.
- **Trigger:** Una *Vulnerabilità* o non conformità può essere corretta tramite una modifica puntuale al codice.
- **Precondizioni:**
 - La *Vulnerabilità* è sufficientemente circoscritta nel codice.
 - L'Agente OWASP può proporre una soluzione locale senza modifiche architeturali.
- **Postcondizioni:** Il *Security Auditor* visualizza uno snippet di codice correttivo da valutare manualmente.
- **Scenario Principale:**
 1. L'Agente OWASP seleziona il frammento di codice vulnerabile.
 2. L'Agente OWASP richiede una versione corretta del frammento.
 3. Il modello genera uno snippet alternativo.
 4. L'Agente OWASP presenta al *Security Auditor* il confronto tra codice originale e codice proposto.
- **Scenario Alternativo:**
 - L'Agente non riceve risposta dall'*LLM* entro il tempo limite oppure riceve un output la cui struttura non è parsabile dal sistema, l'elaborazione viene interrotta e il flusso passa all'UC 5 per la visualizzazione dell'errore.

5.3.5 UC 14: Generazione descrizione testuale della *Remediation*

Descrive la generazione di una spiegazione testuale della correzione da effettuare.

- **Attore Primario:** *Security Auditor / Team Lead / Sviluppatore*.
- **Attori Secondari:** *Repository Git, API Claude Code*.
- **Trigger:** Una *Vulnerabilità* o non conformità non può essere risolta in modo affidabile tramite uno snippet di codice.
- **Precondizioni:**
 - È disponibile una *Vulnerabilità* o non conformità rilevata, l'agente dispone del contesto sufficiente per descrivere il problema.
- **Postcondizioni:** Il *Security Auditor* riceve una descrizione chiara della *Remediation* consigliata.
- **Scenario Principale:**
 1. L'Agente OWASP analizza il problema rilevato.
 2. L'Agente OWASP genera una spiegazione della causa del problema.
 3. L'Agente OWASP descrive le azioni consigliate per mitigare o correggere il problema.
 4. L'Agente OWASP inserisce la descrizione nel report finale.
- **Scenario Alternativo:**
 - L'Agente non riceve risposta dall'*LLM* entro il tempo limite oppure riceve un output la cui struttura non è parsabile dal sistema, l'elaborazione viene interrotta e il flusso passa all'UC 5 per la visualizzazione dell'errore.

6 Agente Changelog: *Changelog* da *User Stories*

6.1 Introduzione all'agente

L'Agente di *Changelog* da *User Stories* funge da mediatore semantico tra il progresso puramente tecnico del team di sviluppo e le necessità informative degli *Stakeholder* di business come il *Project Manager*. In un contesto *Agile_C*, comprendere l'impatto reale di uno *Sprint* può essere complesso per figure non tecniche se i resoconti si limitano a liste di *Commit* o *Ticket* criptici. Questo agente automatizza la traduzione delle attività concluse in report chiari, garantendo che ogni aggiornamento del software sia accompagnato da una narrazione del valore di business generato, senza però tralasciare il dettaglio tecnico necessario alla manutenzione interna.

6.2 Obiettivi e Funzionalità

L'agente si concentra sulla sintesi e trasformazione del progresso di progetto attraverso tre direttrici:

1. **Recupero e Analisi dei Dati:** Integrazione con il sistema di task management *GitHub Issues_C* per l'estrazione automatizzata dei task completati.
2. **Sintesi Semantica Multitarget:** Generazione di report differenziati; uno tecnico (*Changelog* Tecnico) per il team di sviluppo e uno di valore (*Changelog* di Business) per i clienti e il management.
3. **Validazione e Qualità del Contesto:** Monitoraggio della qualità informativa dei task per prevenire la generazione di report ambigui o privi di valore.

6.3 *Casi d'Uso*

Di seguito i *Casi d'Uso* dell'Agente Changelog.

6.3.1 UC 15: Recupero e Filtraggio *User Stories*

Questo caso d'uso riguarda l'estrazione delle attività completate dai sistemi esterni per avviare il processo di analisi.

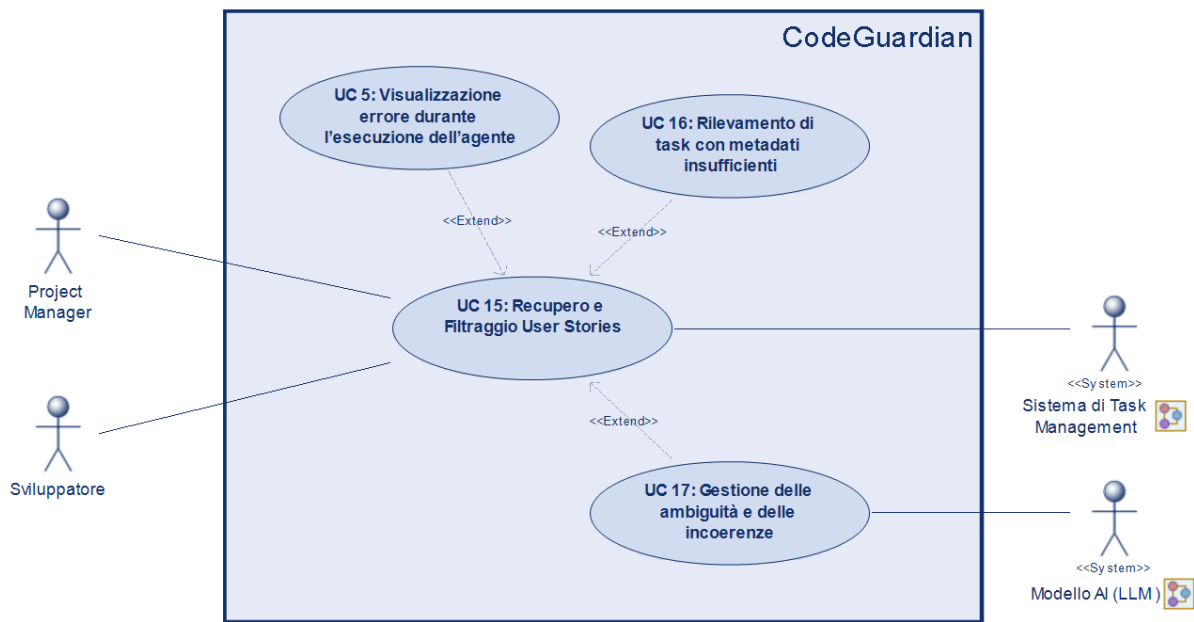


Figura 6: Use Cases Recupero e Filtraggio User Stories

- **Attore Primario:** Sviluppatore / *Project Manager*.
- **Attori Secondari:** Sistema di Task Management *GitHub Issues*.
- **Trigger:** Lo Sviluppatore o il *Project Manager* richiede la generazione del report a conclusione di uno *Sprint*.
- **Precondizioni:**
 - L'Agente Changelog dispone delle credenziali e dei token di accesso validi per le *API* del sistema esterno.
 - È stato specificato un identificativo valido per lo *Sprint* (*Sprint ID*).
 - Le task completate durante lo *Sprint* si trovano in uno stato terminale positivo.
- **Postcondizioni:** L'Agente dispone delle *User Stories* e dei task completati nello *Sprint* selezionato.
- **Scenario Principale:**

1. Il *Project Manager* o lo Sviluppatore seleziona lo *Sprint* da analizzare comunicando lo *Sprint ID*.
2. Il sistema esterno restituisce la lista di tutte le lavorazioni associate.
3. L'Agente filtra la lista, mantenendo esclusivamente le *User Stories* e i task che si trovano in uno stato terminale positivo (es. **Done**, **Closed**).
4. L'Agente estrae da ogni task i campi fondamentali: Titolo, Descrizione, Criteri di Accettazione e Commenti rilevanti.
5. L'Agente mappa i dati in una struttura interna e li memorizza temporaneamente.

- **Scenari alternativi:**

- Se lo *Sprint ID* non esiste o non contiene task completati il sistema di Task Management restituisce un errore o una lista vuota, l'agente interrompe l'esecuzione e il flusso passa all'UC 5.

- **Estensioni:**

- UC 16: Rilevamento di task con metadati insufficienti e segnalazione all'utente.
- UC 17: Gestione delle ambiguità se l'Agente Changelog rileva discrepanze tra titolo e contenuto tecnico.

6.3.2 UC 16: Rilevamento di task con metadati insufficienti

- **Attore Primario:** Sviluppatore / *Project Manager*.

- **Attori Secondari:** Sistema di Task Management.

- **Trigger:** Durante l'esecuzione del recupero (UC 15), l'agente individua una o più *User Stories* prive di descrizione o di criteri di accettazione minimi.

- **Precondizioni:**

- L'Agente ha superato la fase di autenticazione *API* e ha estratto la lista grezza delle task dello *Sprint*.

- **Postcondizioni:** Il flusso di generazione dei report (UC 18 / UC 19) prosegue solo per le task complete di tutti i metadati, mentre l'utente riceve la notifica tempestiva dei dati mancanti da integrare manualmente nei task incompleti.

- **Scenario Principale:**

1. L'Agente analizza il contenuto testuale dei campi informativi di ogni task.
2. L'Agente rileva che una o più task presentano un livello informativo nullo o sotto la soglia minima stabilita dal sistema.

3. L'Agente contrassegna internamente queste task come "Non documentabili per scarsità di dati" ed esclude i relativi ID dal flusso di generazione automatica.
4. Il sistema genera un avviso nel log di esecuzione per informare l'utente delle specifiche task scartate.

6.3.3 UC 17: Gestione delle ambiguità e delle incoerenze

- **Attore Primario:** Sviluppatore / *Project Manager*.
- **Attori Secondari:** Modello AI (*LLM*).
- **Trigger:** Durante la validazione dei dati aggregati nell'UC 15, l'agente rileva una forte discrepanza semantica tra il titolo/descrizione della *User Story* e i dati tecnici correlati (es. log dei *Commit* associati).
- **Precondizioni:**
 - I dati della *User Story* e i dati del *Repository Git* (*Commit*/branch) sono stati recuperati con successo e sono pronti per l'analisi.
- **Postcondizioni:** Viene intercettato un potenziale errore di contesto prima della generazione del report definitivo, garantendo la perfetta coerenza tra ciò che viene dichiarato nel *Changelog* Tecnico e ciò che viene spiegato nel *Changelog* di Business.
- **Scenario Principal:**
 1. L'Agente esegue un confronto semantico automatico tramite *LLM* tra la documentazione della task e l'effettivo codice/messaggio di *Commit* associato.
 2. L'Agente identifica un conflitto (es. task registrata come "Modifica Testi" ma modifiche reali focalizzate sulla "Sicurezza delle API").
 3. L'Agente contrassegna la voce come "Ambigua" all'interno del sistema di orchestrazione.
 4. Il sistema presenta una notifica visiva allo Sviluppatore o al *Project Manager*, richiedendo una risoluzione manuale o una conferma su quale contesto dare priorità (Documentale o Tecnico).

6.3.4 UC 18: Generazione *Changelog* Tecnico (Dev-to-Dev)

L'Agente genera un Changelog Dev-to-Dev partendo dai dati delle User Stories.

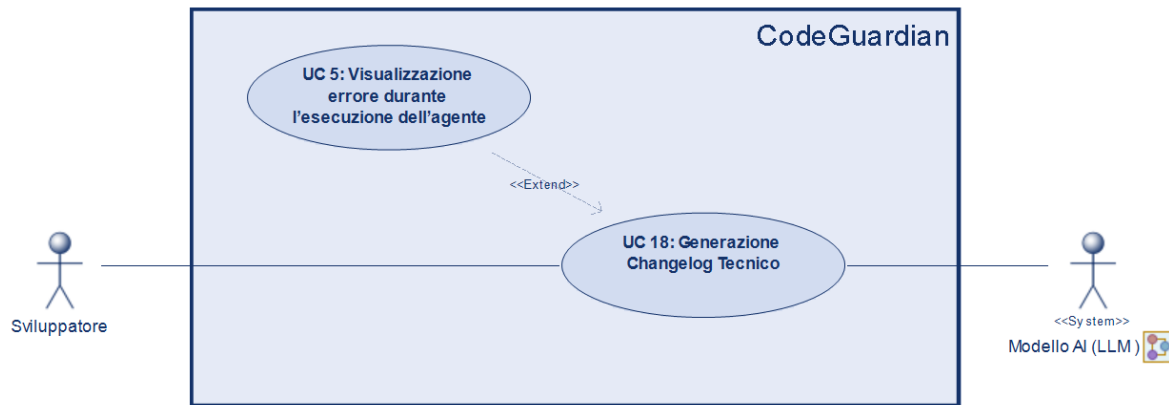


Figura 7: Use Case Generazione Changelog Tecnico

- **Attore Primario:** Sviluppatore.
- **Attori Secondari:** Modello AI (*LLM*).
- **Trigger:** Richiesta esplicita dello sviluppatore di generare un *Changelog* Tecnico e conclusione con successo della fase di recupero, filtraggio e validazione dei dati dello *Sprint* (UC 15).
- **Precondizioni:**
 - L'Agente Changelog dispone del set di dati delle *User Stories* validato e privo di ambiguità bloccanti (superamento di UC 16 e UC 17).
 - I parametri di configurazione del *Repository* sono accessibili.
- **Postcondizioni:** Viene generato un report tecnico pronto per la revisione umana (UC 4) o per l'esportazione diretta (UC 6).
- **Scenario Principale:**
 1. L'Agente analizza il set di dati aggregato e raggruppa le lavorazioni in macro-categorie tecniche standard (es. *Added*, *Changed*, *Fixed*, *Deprecated*).
 2. Per ogni *User Story*, l'Agente interroga l'*LLM* inviando i dettagli tecnici e i criteri di accettazione, richiedendo una sintesi in linguaggio ingegneristico asciutto e privo di ridondanze commerciali.
 3. L'Agente genera la struttura del documento formattando l'output con un formato di *Markdown*.

4. L'Agente inserisce per ogni voce i collegamenti ipertestuali e gli identificativi (ID) corrispondenti ai *Ticket* di origine sul Sistema di Task Management.
5. La bozza del file prodotto viene memorizzata in attesa di essere validata.

- **Scenario Alternativo:**

- L'Agente non riceve risposta dall'*LLM* entro il tempo limite oppure riceve un output la cui struttura non è parsabile dal sistema, l'elaborazione viene interrotta e il flusso passa all'UC 5 per la visualizzazione dell'errore.

6.3.5 UC 19: Generazione *Changelog* di Business (Dev-to-Client)

L'Agente esegue una traduzione semantica per trasformare il gergo tecnico in benefici tangibili per gli *Stakeholder*.

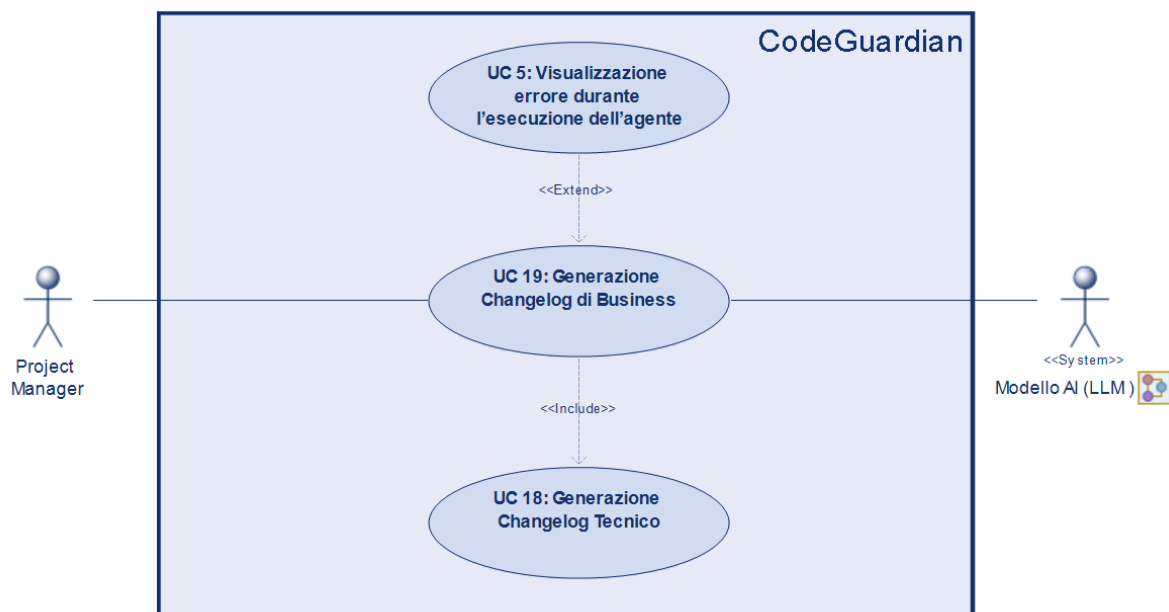


Figura 8: Use Case Generazione Changelog di Business

- **Attore Primario:** *Project Manager*.
- **Attori Secondari:** Modello AI (*LLM*).
- **Trigger:** Richiesta esplicita del *Project Manager* di generare un *Changelog* di Business, dopo la creazione del *Changelog* Tecnico.
- **Precondizioni:**
 - L'Agente Changelog dispone del set di dati delle *User Stories* validato e privo di ambiguità bloccanti (superamento di UC 16 e UC 17).

- I parametri di configurazione del *Repository* sono accessibili.
- **Postcondizioni:** È disponibile un documento di *Changelog* orientato al business pronto per la revisione umana (UC 4) o per l'esportazione diretta (UC 6).
- **Scenario Principale:**
 1. L'Agente riceve in input il testo del *Changelog* Tecnico prodotto dall'UC 18.
 2. L'Agente interroga l'*LLM* inviando il testo tecnico e applicando le direttive sul tono e sull'audience a cui si riferisce il documento.
 3. L'Agente analizza le singole voci tecniche (es. modifiche a database, refactoring, patch di sicurezza) e ne estrae l'equivalente impatto funzionale.
 4. L'Agente traduce i termini specialistici in benefici tangibili per l'utente finale, eliminando i dettagli implementativi di basso livello, poco rilevanti e poco comprensibili per il Cliente.
 5. L'Agente struttura la bozza del report di business e la memorizza, pronta per la validazione umana (UC 4).
- **Scenario Alternativo:**
 - Il Changelog Tecnico non è stato generato in precedenza al passo 1, quindi Il sistema invoca automaticamente la generazione del Changelog Tecnico (Inclusione UC 18).
 - L'Agente non riceve risposta dall'*LLM* entro il tempo limite oppure riceve un output la cui struttura non è parsabile dal sistema, l'elaborazione viene interrotta e il flusso passa all'UC 5 per la visualizzazione dell'errore.
- **Inclusioni:**
 - UC 18: Generazione Changelog Tecnico.

7 Requisiti

I requisiti riportati in questa sezione sono stati derivati dai *Casi d'Uso* descritti nel capitolo 3 e dalle indicazioni emerse dagli incontri con il **Proponente**. In particolare, il perimetro del progetto è stato chiarito come sviluppo di agenti specializzati da integrare in un'architettura già esistente, e non come realizzazione di una nuova architettura completa ad agenti.

7.1 Requisiti funzionali

Codice	Priorità	Descrizione	Fonti
RF.37	Obbligatorio	Il sistema deve consentire a un utente non autenticato di registrarsi creando un nuovo account e inserendo nome, cognome, email e password.	UC 20
RF.38	Obbligatorio	Il sistema deve permettere all'utente, in fase di registrazione, di selezionare il proprio ruolo operativo (Sviluppatore, <i>Security Auditor</i> , <i>Project Manager</i>).	UC 20
RF.39	Obbligatorio	Il sistema deve validare i dati inseriti (formato dell'email, robustezza della password) e mostrare un messaggio d'errore in caso di mancata conformità.	UC 20
RF.40	Obbligatorio	Il sistema deve impedire la registrazione e mostrare un messaggio informativo se l'indirizzo email inserito è già associato a un account esistente.	UC 20
RF.41	Obbligatorio	Il sistema deve consentire a un utente registrato di autenticarsi fornendo le proprie credenziali (email e password) e di conseguenza sbloccare i permessi e le funzionalità associate allo specifico ruolo dell'utente.	UC 21
RF.42	Obbligatorio	Il sistema deve negare l'accesso e mostrare un messaggio di errore generico nel caso in cui le credenziali inserite non siano valide.	UC 21
RF.0	Obbligatorio	Il sistema deve consentire all'utente di inserire, convalidare e memorizzare in modo sicuro i token di accesso personali necessari per l'integrazione con i servizi esterni (<i>Repository Git</i> e Sistema di Task Management).	UC 0

Codice	Priorità	Descrizione	Fonti
RF.1	Obbligatorio	Il sistema deve consentire all'utente di selezionare il <i>Repository Git</i> da utilizzare come contesto di esecuzione degli agenti, specificando anche il riferimento di base desiderato tra branch e <i>Commit</i> specifico.	UC 1
RF.2	Desiderabile	L'utente può utilizzare come riferimento del <i>Repository Git</i> una <i>Pull Request</i> specifica.	UC 1
RF.3	Obbligatorio	Il sistema deve mostrare un messaggio di avviso all'utente qualora il <i>repository</i> o l'ambito di analisi selezionato contenga codice sorgente scritto in linguaggi non supportati.	UC 1
RF.4	Obbligatorio	Il sistema deve consentire all'utente di selezionare l'ambito di analisi, scegliendo uno o più file o directory del Repository.	UC 2
RF.5	Desiderabile	Il sistema deve mostrare un messaggio di errore e impedire l'avvio dell'agente qualora l'ambito di analisi selezionato (numero di file o volume di righe di codice) ecceda i limiti dimensionali entro i quali il modello <i>LLM</i> garantisce performance affidabili.	UC 2, 5
RF.6	Desiderabile	Se supportato dalla configurazione del sistema, il sistema deve consentire la selezione dell'intero Repository come ambito di analisi.	UC 2
RF.7	Obbligatorio	Il sistema deve consentire all'utente di visualizzare il report prodotto dall'agente, includendo almeno criticità rilevate, suggerimenti proposti e file coinvolti.	UC 3
RF.8	Obbligatorio	Il sistema deve consentire all'utente di validare, modificare o rifiutare manualmente una proposta generata da un agente prima della sua eventuale applicazione.	UC 4
RF.9	Obbligatorio	Il sistema deve mostrare un messaggio di errore esplicativo in caso di fallimento durante l'esecuzione di un agente, indicando, ove possibile, una possibile azione correttiva.	UC 5, 15
RF.10	Desiderabile	Il sistema deve bloccare l'esecuzione di un agente e delegare la visualizzazione di un messaggio d'errore qualora la richiesta superi i limiti di utilizzo API imposti.	UC 5

Codice	Priorità	Descrizione	Fonti
RF.11	Desiderabile	Il sistema deve interrompere l'esecuzione di un agente e mostrare un messaggio di errore all'utente qualora il tempo di elaborazione del modello AI (LLM) superi la soglia massima consentita (Timeout di risposta).	UC 5, 8, 9, 12, 13, 14, 18, 19
RF.12	Obbligatorio	Il sistema deve consentire l'esportazione del report generato da un agente in formato PDF.	UC 6
RF.13	Opzionale	Il sistema deve mantenere un archivio storico dei report generati, consentendo all'utente di selezionare un report passato ai fini dell'esportazione.	UC 6
RF.14	Obbligatorio	Il sistema deve generare o aggiornare il file README.md del progetto in modo coerente con lo stato corrente del Repository e in lingua inglese.	UC 7
RF.15	Opzionale	Il sistema deve permettere all'utente di impostare un modello di stile personalizzato per il file README.md.	UC 7
RF.16	Obbligatorio	Il sistema deve generare o aggiornare la documentazione inline del codice (JSDoc) per gli elementi sorgente supportati.	UC 8
RF.17	Obbligatorio	Il sistema deve correggere documentazione inline esistente quando questa non è più coerente con il codice.	UC 8
RF.18	Desiderabile	Il sistema deve segnalare allo Sviluppatore i casi in cui la logica risulti troppo complessa per una documentazione automatica affidabile.	UC 8
RF.19	Obbligatorio	Il sistema deve generare o aggiornare la documentazione delle API del progetto.	UC 9
RF.20	Obbligatorio	Il sistema deve riconoscere quando le modifiche apportate al codice sorgente non alterano il comportamento, i componenti o gli endpoint analizzati, terminando l'esecuzione senza generare nuove proposte di aggiornamento documentale.	UC 7, 8, 9
RF.21	Obbligatorio	Il sistema deve eseguire un'analisi di sicurezza del codice alla ricerca di vulnerabilità riconducibili alle categorie <i>OWASP</i> rilevanti.	UC 10
RF.22	Obbligatorio	Il sistema deve associare ogni vulnerabilità rilevata almeno al file interessato e, se disponibile, alla porzione di codice coinvolta.	UC 10

Codice	Priorità	Descrizione	Fonti
RF.23	Obbligatorio	Il sistema deve classificare le vulnerabilità rilevate indicando almeno categoria <i>OWASP</i> e livello di gravità.	UC 10
RF.24	Obbligatorio	Il sistema deve verificare la conformità del codice rispetto alle policy di sviluppo sicuro definite dal team nel file <i>CLAUDE.md</i> .	UC 11
RF.25	Obbligatorio	Il sistema deve generare suggerimenti di remediation per vulnerabilità o violazioni di policy rilevate.	UC 12
RF.26	Obbligatorio	Quando la problematica è sufficientemente localizzata nel codice, il sistema deve poter generare uno snippet di codice correttivo da sottoporre a validazione umana.	UC 13
RF.27	Obbligatorio	Quando non è possibile proporre in modo affidabile uno snippet locale, il sistema deve generare una descrizione testuale della remediation da effettuare.	UC 14
RF.28	Obbligatorio	Il sistema deve recuperare, da un sistema di Task Management esterno, le User Stories e i task completati relativi a uno specifico Sprint.	UC 15
RF.29	Obbligatorio	Il sistema deve filtrare le User Stories e i task completati relativi a uno specifico Sprint.	UC 15
RF.30	Obbligatorio	Il sistema deve poter identificare lo Sprint di riferimento attraverso la label contenente il codice dello Sprint associata alle issue o ai task.	UC 15, Verbale esterno 2026/04/24
RF.31	Obbligatorio	Il sistema deve estrarre dai task selezionati almeno titolo, descrizione, criteri di accettazione e commenti rilevanti.	UC 15
RF.32	Obbligatorio	Il sistema deve rilevare task con metadati insufficienti e segnalarli all'utente, escludendoli dal flusso di generazione automatica del <i>Changelog</i> .	UC 16
RF.33	Opzionale	Il sistema deve rilevare ambiguità o incoerenze tra la documentazione della User Story e i dati tecnici associati, segnalandole all'utente per risoluzione manuale.	UC 17
RF.34	Obbligatorio	Il sistema deve generare un <i>Changelog</i> tecnico orientato allo sviluppo, strutturato a partire dai task validati dello Sprint.	UC 18

Codice	Priorità	Descrizione	Fonti
RF.35	Desiderabile	Il <i>Changelog</i> tecnico deve includere i collegamenti ipertestuali e gli identificativi dei ticket di origine, quando tali informazioni sono disponibili.	UC 18
RF.36	Obbligatorio	Il sistema deve generare un <i>Changelog</i> orientato al business, comprensibile a stakeholder non tecnici, a partire dal <i>Changelog</i> tecnico, generato dai dati validati dello Sprint.	UC 18,19

7.2 Requisiti qualitativi

Codice	Priorità	Descrizione	Fonti
RQ.6	Desiderabile	Il processo di verifica delle credenziali e autenticazione non deve superare i 2 secondi, al fine di garantire un'esperienza utente fluida.	UC 21
RQ.0	Obbligatorio	Il sistema deve adottare un approccio <i>Human-in-the-loop</i> , mantenendo il controllo umano nella validazione degli output prodotti dagli agenti.	UC 4, Verbale esterno 2026/03/23
RQ.1	Obbligatorio	Gli output prodotti dagli agenti devono risultare utili al miglioramento del codice, ad esempio tramite segnalazioni, suggerimenti testuali, frammenti di codice o proposte di soluzione.	Verbale esterno 2026/03/23
RQ.2	Desiderabile	Il sistema deve minimizzare i falsi positivi nella generazione della documentazione, evitando di proporre aggiornamenti per modifiche al codice puramente stilistiche o marginali.	UC 7, 8, 9
RQ.3	Obbligatorio	Devono essere definite modalità di test e validazione per gli agenti sviluppati e per i modelli o servizi utilizzati.	Verbale esterno 2026/03/23
RQ.4	Obbligatorio	L'output dell'Agente Changelog destinato al cliente o agli stakeholder deve essere scritto in modo comprensibile dal lato business, evitando tecnicismi non necessari.	Verbale esterno 2026/04/24

Codice	Priorità	Descrizione	Fonti
RQ.5	Desiderabile	Il tempo di attesa per l'esecuzione di un agente su un ambito di analisi ristretto (es. file singolo o porzione di codice locale) non dovrebbe superare una soglia temporale ragionevole (es. 2-3 minuti), al fine di non ostacolare il flusso di lavoro dell'utente.	Capitolato C2

7.3 Requisiti di vincolo

Codice	Priorità	Descrizione	Fonti
RV.0	Obbligatorio	Gli agenti sviluppati devono integrarsi in un'architettura già esistente fornita dal Proponente.	Verbale esterno 2026/03/23
RV.1	Obbligatorio	Il progetto non deve prevedere l'addestramento di nuovi modelli, ma l'integrazione di modelli o servizi tramite API.	Verbale esterno 2026/03/23
RV.2	Obbligatorio	Il sistema deve poter operare sui Repository di esempio e sul codice reso disponibile dal Proponente per integrazione e test.	Verbale esterno 2026/03/23
RV.3	Obbligatorio	Il progetto non prevede lo sviluppo dell'Orchestratore né degli agenti dedicati all'aggiornamento delle librerie, alla sostituzione del codice obsoleto al mantenimento della copertura dei test, in quanto già in sviluppo presso Var Group.	Verbale esterno 2026/03/23
RV.4	Obbligatorio	Il sistema deve basarsi sull'analisi di Repository software e sulla produzione di output documentazione e remediation coerenti con il perimetro del progetto Code Guardian.	Capitolato C2
RV.5	Desiderabile	Il sistema dovrebbe prevedere un meccanismo di limitazione di utilizzo a livello di configurazione per tutelare il budget API.	Verbale interno 2026/06/02
RV.6	Obbligatorio	Il sistema deve supportare e garantire l'analisi per Repository contenenti codice sorgente scritto nei linguaggi TypeScript, JavaScript e Python.	Verbale esterno 2026/06/03

7.4 Requisiti di sicurezza

Codice	Priorità	Descrizione	Fonti
RS.5	Obbligatorio	Il sistema deve memorizzare le password degli utenti nel database esclusivamente in formato cifrato, applicando tecniche di hashing.	UC 20
RS.0	Obbligatorio	Il sistema deve memorizzare i token di accesso e le credenziali fornite dall'utente esclusivamente in formato cifrato all'interno del proprio livello di persistenza.	UC 0
RS.1	Obbligatorio	Le eventuali modifiche proposte al codice o ai file del Repository non devono essere applicate senza validazione umana preventiva.	Verbale esterno 2026/03/23
RS.2	Obbligatorio	I risultati dell'analisi di sicurezza devono riportare almeno il tipo di problematica rilevata, il livello di gravità e, se disponibile, il riferimento al file o alla porzione di codice interessata.	UC 10
RS.3	Obbligatorio	L'accesso a Repository, sistemi di Task Management e servizi esterni deve avvenire tramite credenziali, token o chiavi API validi e coerenti con i permessi richiesti dall'operazione.	UC 0, 15
RS.4	Desiderabile	Il sistema dovrebbe interfacciarsi con i servizi LLM esterni utilizzando policy o tier di servizio API che garantiscano che il codice sorgente analizzato non venga trattenuto o utilizzato per l'addestramento di modelli di terze parti.	Capitolato C2

8 Tracciamento

Questa sezione illustra il tracciamento bidirezionale tra le fonti originali (Casi d'Uso, Verbalì, Capitolato) e i requisiti derivati. Le tabelle seguenti mappano questa corrispondenza in modo sistematico, garantendo coerenza e facilitandone la consultazione.

8.1 Tracciamento Fonte - Requisiti

Fonte	Requisiti
UC 1	RF.1
UC 2	RF.4, RF.6
UC 3	RF.7
UC 4	RF.8, RQ.0
UC 5	RF.9
UC 6	RF.12
UC 7	RF.14, RF.15, RF.20, RQ.2
UC 8	RF.16, RF.17, RF.18, RF.20, RQ.2
UC 9	RF.19, RF.20, RQ.2
UC 10	RF.21, RF.22, RF.23, RS.2
UC 11	RF.24
UC 12	RF.25
UC 13	RF.26
UC 14	RF.27
UC 15	RF.28, RF.30, RF.31, RS.3
UC 16	RF.32
UC 17	RF.33
UC 18	RF.34, RF.35
UC 19	RF.36
Capitolato C2	RV.4
Verbale esterno 2026/03/23	RQ.0, RQ.1, RQ.3, RV.0, RV.1, RV.2, RV.3, RS.1
Verbale esterno 2026/04/24	RF.30, RQ.4

8.2 Tracciamento Requisito - Fonti

Requisito	Fonti
RF.1	UC 1
RF.2	UC 1
RF.3	UC 1
RF.4	UC 2
RF.5	UC 2, UC 5
RF.6	UC 2
RF.7	UC 3
RF.8	UC 4
RF.9	UC 5, UC 15
RF.10	UC 5
RF.11	UC 5, UC 8, UC 9, UC 12, UC 13, UC 14, UC 18, UC 19
RF.12	UC 6
RF.13	UC 6
RF.14	UC 7
RF.15	UC 7
RF.16	UC 8
RF.17	UC 8
RF.18	UC 8
RF.19	UC 9
RF.20	UC 7, UC 8, UC 9
RF.21	UC 10
RF.22	UC 10
RF.23	UC 10
RF.24	UC 11
RF.25	UC 12
RF.26	UC 13
RF.27	UC 14
RF.28	UC 15
RF.30	UC 15, Verbale esterno 2026/04/24
RF.31	UC 15
RF.32	UC 16
RF.33	UC 17

Requisito	Fonti
RF.34	UC 18
RF.35	UC 18
RF.36	UC 19
RQ.0	UC 4, Verbale esterno 2026/03/23
RQ.1	Verbale esterno 2026/03/23
RQ.2	UC 7, UC 8, UC 9
RQ.3	Verbale esterno 2026/03/23
RQ.4	Verbale esterno 2026/04/24
RV.0	Verbale esterno 2026/03/23
RV.1	Verbale esterno 2026/03/23
RV.2	Verbale esterno 2026/03/23
RV.3	Verbale esterno 2026/03/23
RV.4	Capitolato C2
RS.1	Verbale esterno 2026/03/23
RS.2	UC 10
RS.3	UC 15